Chapter 19: Kotlin/Java Interoperability

By Ellen Shapiro

Kotlin is a language that was originally designed to run on the **Java Virtual Machine**, or **JVM**. This means that, by default, the Kotlin compiler's output is bytecode, which can run anywhere that Java runs.

Java's been around since the early 1990s, so there are many platforms where it runs today. Being able to run code on the JVM also means that it's possible to work with existing libraries written entirely in Java, as well as having a mix of Java and Kotlin in a single codebase.

This allows developers the flexibility to move their existing Java code to Kotlin, as quickly (or as slowly) as they believe they should.

Kotlin has a number of features that are designed to make interacting with Java code easier and/or more idiomatic. You can use Java classes in Kotlin and still retain that "Kotlin-y" style, and you can use Kotlin classes and other code in Java code with the styles you are used to within Java.

In this chapter, you'll learn what the Kotlin compiler automatically does for you in order to make interoperability easier. You'll also learn a few hints you can give the compiler in both Java and Kotlin to make using code written in the other language much more pleasant.

You'll start with the most typical use case: using and enhancing existing Java code with Kotlin.

Mixing Java and Kotlin code

There are several things that the Kotlin compiler will automatically do for you in order to make using code written in Java feel more at home when called from Kotlin, as well as interacting with code written in Kotlin from Java. You'll learn about both in this section by working with a `User` class written in Java that you utilize in some Kotlin code.

Getters and setters

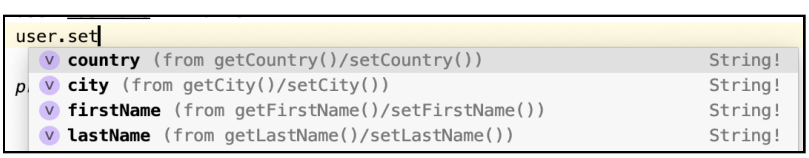
To start, open up the starter project for this chapter and go to the **User.java** file. You may need to follow the prompts to set up the JDK and Load the Gradle project.

You'll see a typical Java class, which uses `private` backing variables and only exposes access to them through `get` and `set` methods for anything outside the current class. Next, go to **main.kt**, and delete the `println` statement. Create a new user instead:

```
val user = User()
```

Notice that, even though this is a Java class, the syntax to create a new object is the same as it is for pure Kotlin objects. You do not include the `new` keyword since it's not needed in Kotlin to create a new instance.

Next, go to a new line, and start typing `user.set`. Instead of the explicit `set...` calls that you saw in the **User.java** file, you'll see property names which are generated by the Kotlin compiler:



Notice that the IDE also displays the names of the methods, from where it's synthesizing these property names.

It also indicates that the type is `String!` — a `String` that is assumed to be non-null. You'll see how to add proper support for nullability later in this chapter.

In `main()` within **main.kt**, add some details for the user:

```
user.firstName = "Bob"
user.lastName = "Barker"
user.city = "Los Angeles"
user.country = "United States"

println("User info:\n$user")
```

Run **main.kt** using the Play button in the upper-left of the Editor panel by the `main()` declaration, and it will print the following:

```
User info:
Bob Barker
Los Angeles, United States
```

Wow, that's nicely formatted! Why is that? If you open **User.java**, you'll notice that `toString()` is overridden at the bottom of the file.

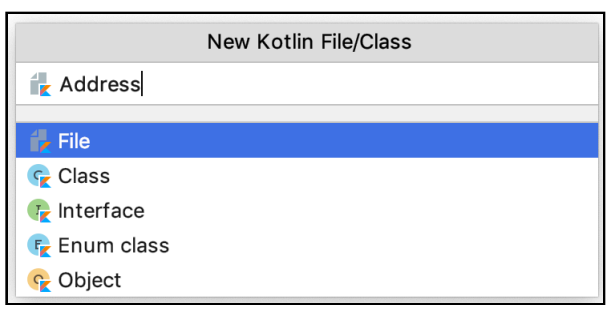
Whenever `toString()` is overridden in Java, Kotlin's string interpolation syntax will call through to that method when using the `$variableName` syntax.

Right now, the `User` class only knows about the user's city and country. What if you wanted to be able to support a full address?

In theory, you could keep adding more properties to `User`, but for the sake of separation of concerns, you probably want to create a new class. And the fun part is, you can use Kotlin to create your new class, and then use it directly from your Java `User` class!

Adding a Kotlin class as a Java property

In IntelliJ IDEA's menu, select **File ▸ New ▸ Kotlin File/Class**. Name the file you're creating **Address**:



Open the resulting **Address.kt** file, which will be empty. Since there are several possible types of addresses, start by adding an `enum class` to handle those different types:

```
enum class AddressType {
  Billing,
  Shipping,
  Gift
}
```

Next, add a `data class`with a constructor to handle creating a new `Address` with the appropriate properties:

```
data class Address(
    val streetLine1: String,
    val streetLine2: String?,
    val city: String,
    val stateOrProvince: String,
    val postalCode: String,
    var addressType: AddressType,
    val country: String = "United States"
) {
  // TODO
}
```

You'll also want to be able to have a nicely formatted address for when you need to send this user mail or ship them something. Replace the `TODO` with a function to do that:

```
fun forPostalLabel(): String {
  var printedAddress = streetLine1
  streetLine2?.let { printedAddress += "\n$it" }
  printedAddress += "\n$city, $stateOrProvince $postalCode"
  printedAddress += "\n${country.toUpperCase()}"
  return printedAddress
}
```

Next, go back to **main.kt**. Delete the two lines where you assign the user's city and country.

Underneath where you printed the user information, add code to create and then print out an address:

```
val billingAddress = Address("123 Fake Street",
    "4th floor",
    "Los Angeles",
    "CA",
    "90291",
    AddressType.Billing)

println("Billing Address:\n$billingAddress\n")
```

Run **main.kt** and you'll see the automatically generated details for the `address` class at the bottom of the console:

```
Billing Address:
Address(streetLine1=123 Fake Street, streetLine2=4th floor,
city=Los Angeles, stateOrProvince=CA, postalCode=90291,
addressType=Billing, country=United States)
```

While this is helpful, it can be a bit difficult to parse visually. In **Address.kt**, override the `toString` function in order to make the printing look a little better:

```
override fun toString(): String {
    return forPostalLabel()
}
```

Run **main.kt** again, and it'll look a little nicer:

```
Billing Address:
123 Fake Street
4th floor
Los Angeles, CA 90291
UNITED STATES
```

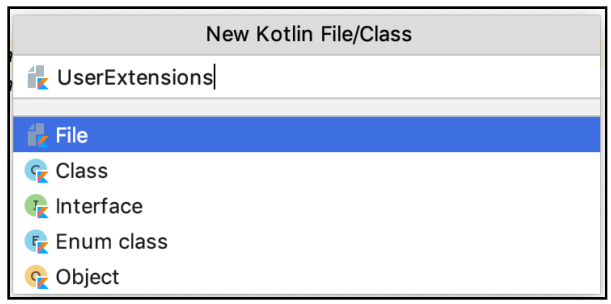
Now that you've moved the information for city and country over to the `Address` class, the properties on the `User` class are no longer necessary. In **User.java**, delete city and country member variables, along with their getters and setters. Update the `toString()` function to:

```
@Override
public String toString() {
  return firstName + " " + lastName;
}
```

Combining the first and last names like this seems like a useful thing for which to add a `get()`-only property — and, instead of doing it in the existing Java class, you can create a Kotlin extension to let you centralize this code to Kotlin.

Adding extension functions to a Java class

Go to **File ▸ New ▸ Kotlin File/Class**. Name your new file **UserExtensions**:



Once the **UserExtensions.kt** file is created, open it and add an extension on the `User` class with your `get()`-only property:

```
val User.fullName: String
  get() = "$firstName $lastName"
```

Now that you have this simplified property, you can go back to **User.java** and update the `toString()` method to call into the extension you just wrote:

```
@Override
public String toString() {
  return UserExtensionsKt.getFullName(this);
}
```

Note that the current user is passed as a parameter, since extensions don't exist in Java. This allows Java functions with potentially clashing names to continue to work correctly since the methods are not being called directly on the object.

By default, unless you provide another name, Kotlin exposes the full file name of a file with extensions or free functions as a wrapper class named `FileNameKt`, and each of those extension methods or free functions as static methods on that wrapper.

That default file naming leaks the fact that you're working with Kotlin in a particular place. If you **want** to know at the call site that you're calling into Kotlin code, then that's fine.

However, if you'd rather have a cleaner name when your Kotlin code comes into Java, you can take advantage of **annotations** to be able to make your file name a bit clearer.

At the top of **UserExtensions.kt**, add the following line:

```
@file:JvmName("UserExtensions")
```

This annotation tells the Kotlin compiler that when creating the Java interop definitions for this file, it should name the wrapper class `UserExtensions` rather than `UserExtensionsKt`.

Now, go back to **User.java** and you can update the `toString()` method to use the name you've set up as the `JvmName`:

```
return UserExtensions.getFullName(this);
```

Neat! There is one pretty significant limitation to what you can do with Kotlin extensions that you may remember from Chapter 13: "Properties": You can't add additional properties with backing fields to them - only properties with custom accessors.

For our next trick, you'll want to scroll back up in **User.java** and add a new property to hold the list of addresses belonging to a user below the other properites:

```
private List<Address> addresses = new ArrayList<>();
```

And then add the corresponding getter and setter below the other getters and setters:

```
public List<Address> getAddresses() {
  return addresses;
}

public void setAddresses(List<Address> addresses) {
  this.addresses = addresses;
}
```

Finally, update `toString()` so you can see how many addresses a user has at a glance in the console:

```
return UserExtensions.getFullName(this) +
    " - Addresses: " + addresses.size();
```

Run **main.kt**, and at the top of the console you'll now see:

```
User info:
Bob Barker - Addresses: 0
```

Now that the backing property `addresses` is set up, you can go back to using Kotlin to work with this backing variable. In **UserExtensions.kt**, add an extension function to get the address of a given type or to return `null` if it doesn't exist:

```
fun User.addressOfType(type: AddressType): Address? {
  return addresses.firstOrNull { it.addressType == type }
}
```

This extension function can be called from either Java or Kotlin but, under the hood, it takes advantage of Kotlin's functional programming and nullability handling. Cool!

You can also add functions that handle validation for adding and removing items from the list of addresses.

In this case, you really only would want one address of a given type — Shipping, Billing or Gift. Add an extension function which adds or updates an address:

```
fun User.addOrUpdateAddress(address: Address) {
  val existingOfType = addressOfType(address.addressType)

  if (existingOfType != null) {
    addresses.remove(existingOfType)
  }

  addresses.add(address)
}
```

Now that all this functionality has been added to the `User` class, it's time to use it in Kotlin! Go to **main.kt**, and below the `println` statement printing out the address, add:

```
user.addOrUpdateAddress(billingAddress)
println("User info after adding address:\n$user")
```

Run **main.kt**, and at the end of the console you'll see:

```
User info after adding address:
Bob Barker - Addresses: 1
```

Now, try to add another address. In **main.kt**, delete the last `println` statement and replace it with:

```
val shippingAddress = Address("987 Unreal Drive",
    null,
    "Burbank",
    "CA",
    "91523",
    AddressType.Shipping)

user.addOrUpdateAddress(shippingAddress)

println("User info after adding addresses:\n$user")
```

Run **main.kt**, and you'll now see at the bottom of the console:

```
User info after adding addresses:
Bob Barker - Addresses: 2
```

Great! Since you have billing and shipping addresses, both are being added. Now, check if the validation you added is working. Update the `AddressType` of `shippingAddress` in the constructor to `AddressType.Billing`.

Run **main.kt** again, and voila!:

```
User info after adding addresses:
Bob Barker - Addresses: 1
```

Since both addresses are showing up as billing address, when the second one is added, it replaces the first one. You've now added a new address to a Java `User`, but done so using validation completely in Kotlin. Congratulations!

> **Note**: Remember to change the `AddressType` of `shippingAddress` back to `AddressType.Shipping` for the next few steps.

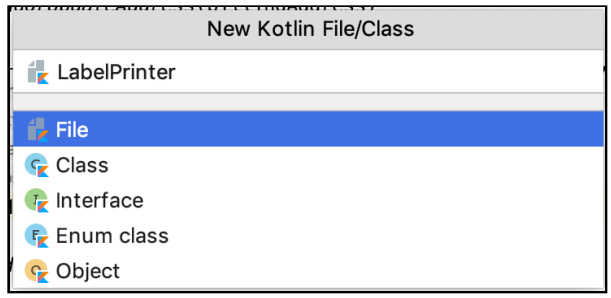
Now that you've been able to use extension functions from Java, it's time to see how free functions in Kotlin work with Java code.

Free functions

Free functions in Kotlin are functions that don't extend any existing class and are not tied to a class themselves. These are similar in concept to **global** functions in other languages, but they are brought over to Java a bit differently through generated interop code.

To demonstrate this, you're going to make a file with free functions to allow you to print a full mailing label for a given address.

Go to **File ▸ New Kotlin File/Class**, and name your file **LabelPrinter**:



Open the resulting **LabelPrinter.kt** file, and add the following code:

```
// 1
fun labelFor(user: User, type: AddressType): String {
  // 2
  val address = user.addressOfType(type)
  if (address != null) {
    // 3
    var label = "-----\n"
    label += "${user.fullName}\n${address.forPostalLabel()}\n"
    label += "-----\n"
    return label
  } else {
    return "\n!! ${user.fullName} does not have a $type address
set up !!\n"
  }
}

// 4
fun printLabelFor(user: User, type: AddressType) {
    println(labelFor(user, type))
}
```

What's happening in this code:

1. You built a free function to create a label string based on a user and a given type of address.
2. Since you're in Kotlin, you can use the `addressOfType` extension method directly on your `User` object to see if an address of the given type exists.
3. You built up a `String` across multiple lines. In Kotlin, this is as simple as making a `var`, then using the += operator to concatenate strings. This is in sharp contrast to the more painful method in Java, which you'll see shortly.
4. A convenience free function to simply print the label generated by the other free function in this file has been added.

To see how this works in Kotlin, go to **main.kt** and add the following lines:

```
println("Shipping Label:")
printLabelFor(user, AddressType.Shipping)
```

Note that you didn't have to do anything involving the `LabelPrinter` here, since the functions you added to that file are available globally to all your Kotlin code in the project. Run **main.kt** again, and you'll see at the bottom of the console:

```
Shipping Label:
-----
Bob Barker
987 Unreal Drive
Burbank, CA 91523
UNITED STATES
-----
```

OK, that was the easy part. Now for the more cumbersome part.

In **User.java**, add a new method to build up a `String` with all the various types of address label:

```
public String allAddresses() {
  StringBuilder builder = new StringBuilder();
  for (Address address : addresses) {
    builder.append(
      address.getAddressType().name() + " address:\n");
    builder.append(
      LabelPrinterKt.labelFor(this, address.getAddressType()));
  }

  return builder.toString();
}
```

You'll notice two major similarities of the free functions files to the way an extension file works in interop here.

Firstly, a wrapper class called `LabelPrinterKt` is generated with the free functions added as static methods, rather than the functions simply being made global. Note that since you are not extending anything, no additional parameters are generated which need to be passed in.

Secondly, the automatically generated name for the wrapper class in a file with only free functions is of the format `FileNameKt`.

Using the same annotation as you did for `UserExtensions`, update the display name of **LabelPrinter.kt** by adding the following at the top of the file:

```
@file:JvmName("LabelPrinter")
```

Now, when you go back to **User.java**, you can update the `allAddresses()` method to use the updated file name:

```
builder.append(
  LabelPrinter.labelFor(this, address.getAddressType()));
```

Now, update the `toString()` method on `User` to take advantage of this functionality:

```
return UserExtensions.getFullName(this) +
    " - Addresses: " + addresses.size() + "\n" +
    allAddresses();
```

Finally, go back and re-run **main.kt**. The output in the console where you were previously just getting how many addresses the user had (and above the "shipping label" output) will be:

```
User info after adding addresses:
Bob Barker - Addresses: 2
Billing address:
-----
Bob Barker
123 Fake Street
4th floor
Los Angeles, CA 90291
UNITED STATES
-----
Shipping address:
-----
Bob Barker
987 Unreal Drive
```

```
Burbank, CA 91523
UNITED STATES
-----
```

Now that you've got all this information about mixing Java and Kotlin code, it's time to look into another piece of the puzzle when bringing Java code into Kotlin: nullability.

Java nullability annotations

Though Java 8 introduced `Optional` to make null values safer to work with, annotations are the way to go when handling nullability between Kotlin and Java.

JetBrains, the makers of the IntelliJ IDEA IDE you've been using throughout this book, have created annotations that you can add to classes, parameters and methods.

These annotations allow you to indicate to JVM languages that have first-class support for nullability (like Kotlin) whether a given object is supposed to be nullable or not.

At the bottom of **main.kt**, add new code to access how many addresses a second user has:

```
val anotherUser = User()
println("Another User has ${anotherUser.addresses.count()}
addresses")
```

Run **main.kt**, and at the end of the console will be:

```
Another User has 0 addresses
```

If you recall, in **User.java**, you initialized the `addresses` variable with an empty `ArrayList` — so this should theoretically never be null. But what happens if you explicitly make `addresses` null?

Update **main.kt** to add a new line between the `anotherUser` creation and the `println` statement immediately after it:

```
anotherUser.addresses = null
```

Run **main.kt**, and you'll get a runtime error:

```
Exception in thread "main" java.lang.IllegalStateException: anotherUser.addresses must not be null
    at MainKt.main(main.kt:36)
```

anotherUser.addresses must not be null

This is because the `anotherUser.addresses.count()` is expecting `addresses` to never be `null`, because nobody's ever told it that it could be `null`.

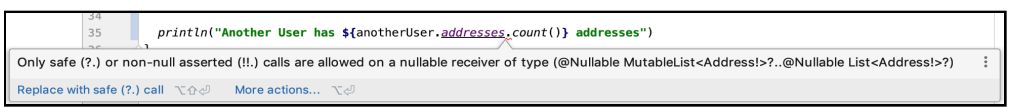
By default, Kotlin code that is generated from Java uses the `!` type for all variables unless otherwise annotated. This means that, unless you specifically tell the compiler something could be `null`, it'll assume it's supposed to be there and either error out or crash if it's not there.

You can avoid this behavior using annotations. Go back to **User.java** and add an explicit annotation to the getter to indicate this property could be `null`:

```
@Nullable
public List<Address> getAddresses() { ... }
```

> **Note**: If offered an option of which version of `@Nullable` to use, select the `org.jetbrains.annotations` one rather than any created by other vendors.

Now, when you go back to **main.kt**, the compiler will give an error, which forces you to validate nullability:

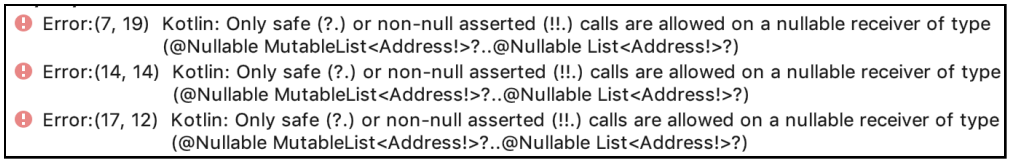


Only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver

Update the line with the error squiggle to add the safe call operator `?.` — indicating that, if something in the chain is `null`, the whole thing should return `null` instead of throwing an exception:

```
println("Another User has ${anotherUser.addresses?.count()}
addresses")
```

Try to run **main.kt** again, and you'll see a few errors in **UserExtensions.kt**:



Only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver

Update these three calls to use the same safe call operator style `addresses?.function()`. When the errors are gone, run **main.kt** again and, this time, the bottom of the console will print:

```
Another User has null addresses
```

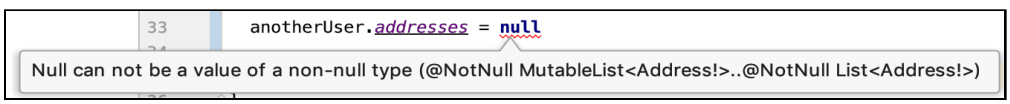
No crashes! Hooray! But what if you want to take a step beyond not crashing: What if you want to actively prevent a caller from actually setting the `addresses` property, which is set up when the class is set up, to `null`?

Because Kotlin takes the getter and the setter and synthesizes them into properties, you only need to annotate the getter to acheive this!

Go back to **User.java** and update the annotation on the `getAddresses` method:

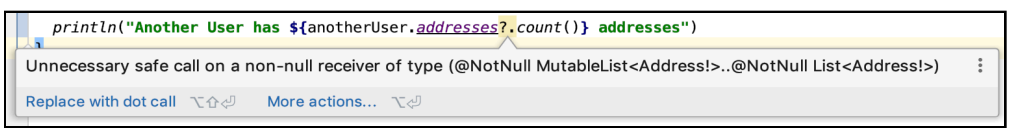
```
@NotNull
public List<Address> getAddresses() { ... }
```

Now, go back to **main.kt** and you'll notice that, even though you didn't annotate the setter, you're now getting an error about trying to set `addresses` to `null`:



Null can not be a value of a non-null type

Comment out the line setting addresses to `null` and that error will go away. You'll now have four warnings about having an unnecessary safe call on a non-null receiver at the four places you added safe call chaining:



Unnecessary safe call on a non-null receiver

While the project will still build if you have these warnings, it's definitely better to go back and remove them since they're no longer necessary.

Since you really never want any of these properties to be `null`, go to **User.java** and add `@NotNull` annotations above all the other two getters:

```
@NotNull
public String getFirstName() { ... }
...
@NotNull
public String getLastName() { ... }
```

In **main.kt**, add a final line printing out the user's first name:

```
println("Another User first name: ${anotherUser.firstName}")
```

Run **main.kt**, and it'll print out:

```
Another User has 0 addresses
Another User first name: null
```

Wait, what? That's supposed to never be `null`! What's happened here?

Because you haven't actually set a default value for the user's first name the way that you have with the empty addresses `ArrayList`, and you haven't assigned a value for `firstName`, that backing variable is still `null`.

In older versions of IntelliJ and Kotlin, this would actually cause a crash at runtime, since you'd made a promise by using the `NotNull` annotation that the value would never be null.

The good news is that Kotlin no longer blindly assumes that whoever's adding nullabilty annotations to Java code knows what they're doing, so it no longer crashes if that person was wrong.

This is particularly something to watch out for when working with other people's code that has Java nullability annotations.

If you find that the code you wrote made incorrect assumptions about nullability, that's easy to fix. However, you won't be able to fix code you've brought in from other sources, like Android OS code or other framework code.

When working with your own Java code, you need to remember that if you don't provide an initial value for a variable, the variable is nullable and should be annotated as such.

Go back to **User.java** and update the nullability annotations on the `firstName` and `lastName` getters to reflect this:

```
@Nullable
public String getFirstName() { ... }
...
@Nullable
public String getLastName() { ... }
```

Run **main.kt** again and you'll see the same printout:

```
Another User first name: null
```

If you think this looks a little silly, now that you've annotated `getFirstName` as nullable, you can use the Elvis operator to print a more reasonable message. Update the last line of **main.kt** to do this:

```
println("Another User first name: ${anotherUser.firstName ?:
"(not set)"}")
```

Run **main.kt** one last time, and you'll see Elvis in action:

```
Another User first name: (not set)
```

Whew! You've done a lot of interesting stuff calling Java from Kotlin code and a tiny bit of calling Kotlin from Java. But there are a few more things you can do to make your code in Kotlin a bit easier to use in Java.

Making your Kotlin Code Java-friendly

In the starter project, there's also a **main.java** file with a `JavaApplication` class. Open this file and replace the `System.out.println()` command with some new code:

```
// 1
User user = new User();
// 2
user.setFirstName("Testy");
user.setLastName("McTesterson");

// 3
Address address = new Address(
  "345 Nonexistent Avenue NW",
  null,
  "Washington",
```

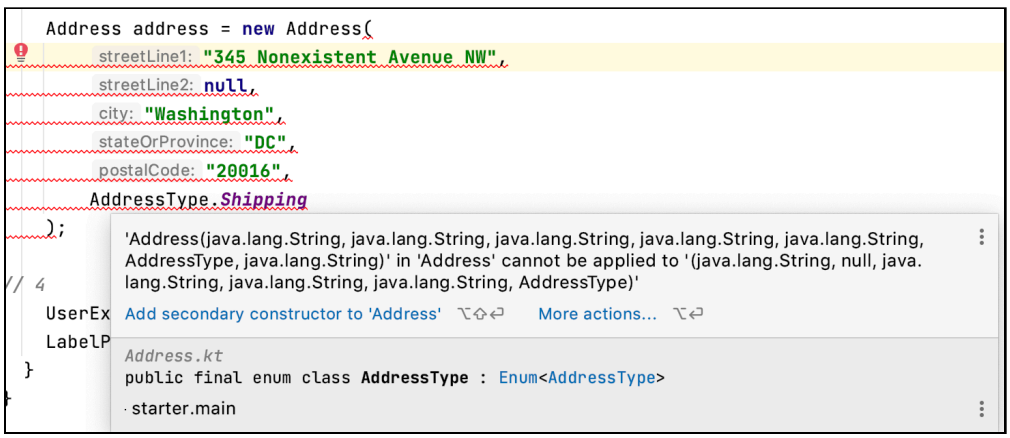
```
    "DC",
    "20016",
    AddressType.Shipping
  );

  // 4
  UserExtensions.addOrUpdateAddress(user, address);
  LabelPrinter.printLabelFor(user, AddressType.Shipping);
```

What's happening in the code you've added, here?

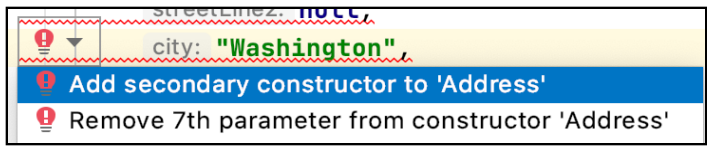
1. You create a new user using traditional Java syntax. Don't forget the `new` keyword and the semicolon at the end when creating new objects in Java!
2. While Kotlin is able to use synthesized property access, Java isn't without some specific helpers you'll see later, so to set up the user's first and last names, you must add them using the explicit setters.
3. Here, you're calling into the Kotlin constructor, which is created by default for `Address` (you'll take a look at the error in a second).
4. You're using an extension method and a free function (both defined in Kotlin) to add the address to the user and then print a label for them.

The error in the `Address` constructor is a little verbose:



Address constructor cannot be applied to types

But if you look at the hints on the error, you can get a better idea of what's gone wrong here:



Remove 7th parameter from constructor 'Address'

The hint is asking if you want to get rid of the seventh parameter from the constructor. This makes sense since you've only passed in six items.

If you go back to **Address.kt**, you'll see the country parameter is the 7th parameter, but it has a default value provided. So why isn't that default value showing up?

This is because, by default, the Kotlin compiler only generates a single constructor with all available parameters. Java doesn't support default parameters in the same way that Kotlin does. If you want to allow default values, you have to create constructors that do not include the various default values.

That sounds really boring. Fortunately, the Kotlin compiler can be told to do this for you! Go to **Address.kt** and update the declaration of the `Address` class:

```
data class Address @JvmOverloads constructor
```

The `@JvmOverloads` annotation tells the compiler that it should generate those boring extra methods for you which omit any parameters that have default values.

In Kotlin, the `constructor` keyword is implied in the declaration of the class when you're adding a bunch of `val` and `var` declarations that can be passed in when the object is created.

However, if you want to add `@JvmOverloads` for the default constructor, you have to explicitly use the `constructor` keyword in order for the proper overloaded constructors to be generated.

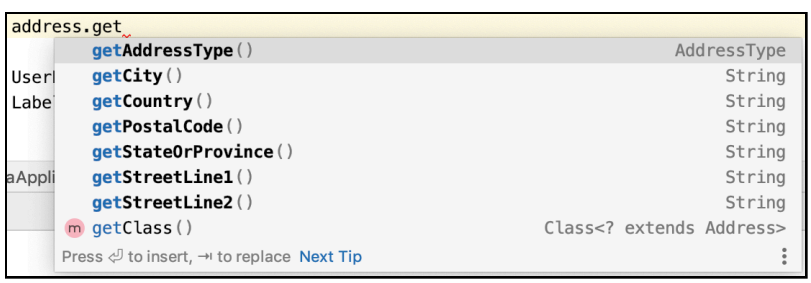
Go back to **main.java**, and your build error should now be resolved. Run the Java program using the Play button in the upper-left corner next to the `main()` method.

When you do, the following will print out:

```
-----
Testy McTesterson
345 Nonexistent Avenue NW
Washington, DC 20016
UNITED STATES
-----
```

> **Note**: As you continue through the chapter, make sure to use the Play button in the upper-left of either the **main.kt** or **main.java** files rather than the Play button at the top-right of the IDE window or the one in the **Run** tab at the bottom of the IDE. The button and the tab on the IDE will run whichever program you ran most recently, Java or Kotlin, whereas the one in the top-left of each file will always run the program in that specific file.

Next, in **main.java**, directly under where the address is constructed, start typing `address.get` and you'll see something cool in the auto-complete offered by IntelliJ:



Those getters that were so annoying to write in Java have been automatically generated by the Kotlin compiler for the `Address` class. Sweet! What about the setters?

Start typing `address.set` and you'll notice something:

Only one setter was automatically generated for the `Address` properties. Why?

Recall that, in Kotlin, anything which is a `val` is supposed to be set by the constructor and then never changed. This means that a setter is completely unnecessary and, therefore, won't be generated.

The only property of `Address` that is a `var` property is the `AddressType`, so that's the only Java setter that gets generated.

Let that setter auto-complete and update the address type to `Billing`:

```
address.setAddressType(AddressType.Billing);
```

Run **main.java**. Since you're trying to print the label for a Shipping address but you've changed it to a Billing address, you'll see:

```
!! Testy McTesterson does not have a Shipping address set up !!
```

You're trying to print a Shipping address label, but you've changed the type on `address` to Billing!

Delete the line setting the address type and run **main.java** again. You will again see:

```
_____
Testy McTesterson
345 Nonexistent Avenue NW
Washington, DC 20016
UNITED STATES
_____
```

Since you're probably going to print Shipping labels, it makes sense to add a default value to the functions in **LabelPrinter.kt**. Open up that file, and add a default `type` value to the `printLabelFor()` function:

```
fun printLabelFor(user: User, type: AddressType =
AddressType.Shipping) {
```

Now, try to use that in Kotlin by going to **main.kt** and updating the `printLabelFor()` line to remove the type:

```
printLabelFor(user)
```

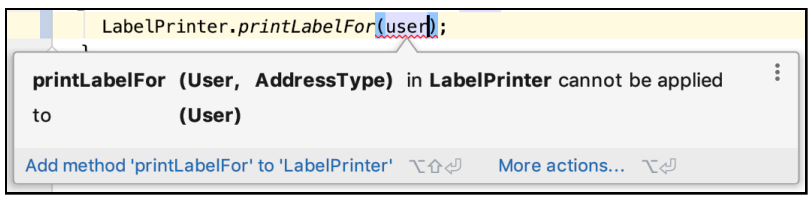
Run **main.kt**, and you'll see the same data print out as before:

```
Shipping Label:
_____
Bob Barker
987 Unreal Drive
Burbank, CA 91523
UNITED STATES
_____
```

That was easy. Next, update the line printing a label at the end of **main.java** to remove the parameter, which has a default value:

```
LabelPrinter.printLabelFor(user);
```

When you do this, you'll see an error:



Similarly to the `constructor`, without a hint, this function doesn't know it needs to generate multiple Java methods to account for anything that has a default value for a given parameter. Time to give it that hint!

In **LabelPrinter.kt**, add the `@JvmOverloads` annotation directly above where `printLabelFor()` is declared:

```
@JvmOverloads
fun printLabelFor(user: User, ....
```

Now, go back to **main.java**, and the error should be resolved. Run **main.java**, and the output should be the same as it was previously:

```
-----
Testy McTesterson
345 Nonexistent Avenue NW
Washington, DC 20016
UNITED STATES
-----
```

Now, it's time to learn how to deal with class non-companion objects in Java!

Accessing nested Kotlin objects

In Kotlin, you can create objects that are not necessarily classes within a class. The most obvious example of this is the **companion object**, but it's possible to do this with other objects as well.

How do you access these in Java? Let's turn an `Address` into a `HashMap`, which can eventually be turned into JSON to be sent back and forth to a server, to find out.

In **Address.kt**, below the last function but still within the `Address` class, add an `object` with the JSON keys you'll be using to distinguish between the items in the `HashMap`:

```
object JSONKeys {
  val streetLine1 = "street_1"
  val streetLine2 = "street_2"
  val city = "city"
  val stateOrProvince = "state"
  val postalCode = "zip"
  val addressType = "type"
  val country = "country"
}
```

Next, go to **main.java** and attempt to access the keys that you've just created:

```
Address.JSONKeys keys = Address.JSONKeys;
```

You'll get the following error:

Expression expected

What this error is rather obtusely trying to tell you is that, when using interoperability with a nested Kotlin object, there needs to be an instance of that object to work with before doing anything.

Fortunately, Kotlin generates a Java `INSTANCE` variable, which can be accessed on non-companion nested objects for Java access.

In **main.java**, use the `INSTANCE` generated for a non-companion object to grab a reference to the keys:

```
Address.JSONKeys keys = Address.JSONKeys.INSTANCE;
```

And the error should now be gone. Now, add code to create a `HashMap`, which is an object with keys and values:

```
HashMap<String, Object> addressJSON = new HashMap<>();
addressJSON.put(
  keys.getStreetLine1(), address.getStreetLine1());
```

This code creates the `HashMap` in Java with the appropriate types, then uses the `put()` method to add a key and its value. However, this is a bit verbose.

The getter and setter methods create a lot of noise. What if you'd prefer not to use them? Good news! You can tell the Kotlin compiler to not to create these methods on properties of a class with (you guessed it!) an annotation.

Go back to **Address.kt** and update the `vals` and `vars` in the `constructor` to use the `@JvmField` annotation:

```
data class Address @JvmOverloads constructor (
    @JvmField val streetLine1: String,
    @JvmField val streetLine2: String?,
    @JvmField val city: String,
    @JvmField val stateOrProvince: String,
    @JvmField val postalCode: String,
    @JvmField var addressType: AddressType,
    @JvmField val country: String = "United States") {
```

The `@JvmField` annotation tells Kotlin that for JVM languages, it doesn't need to generate getter and setter methods — it just creates a **field**, or a property variable, and uses that directly. The `@JvmField` annotation can be used on all sorts of types, and is generally advisable to use on properties of a Kotlin object if you wish to avoid generating getters and setters, no matter what their type.

For constants that are basic types on independent Kotlin objects, such as `String` and `Int`, you can use the `const` keyword in Kotlin to achieve the same effect without littering your code with `@` symbols.

Scroll down to the `JSONKeys` object and update each variable to use `const`:

```
object JSONKeys {
  const val streetLine1 = "street_1"
  const val streetLine2 = "street_2"
  const val city = "city"
  const val stateOrProvince = "state"
  const val postalCode = "zip"
  const val addressType = "type"
  const val country = "country"
}
```

Now, you can go back to **main.java**; update the first line that you've already added to use the field and `const` you've defined.

```
addressJSON.put(keys.streetLine1, address.streetLine1);
```

That looks a little nicer! Now, add code to put the rest of the properties for the address into the `HashMap`, then print the `HashMap` out:

```
addressJSON.put(keys.streetLine2, address.streetLine2);
addressJSON.put(keys.city, address.city);
addressJSON.put(keys.stateOrProvince, address.stateOrProvince);
addressJSON.put(keys.postalCode, address.postalCode);
addressJSON.put(keys.country, address.country);
addressJSON.put(keys.addressType, address.addressType.name());

System.out.println("Address JSON:\n" + addressJSON);
```

Try to run **main.java**, and you'll see a couple errors in **User.java** — since you've told the compiler it doesn't need to use getter and setter methods, the methods you were using to get the address type are now gone.

Replace the getters causing the errors with field access:

```
builder.append(address.addressType.name() + " address:\n");
builder.append(LabelPrinter.labelFor(this,
address.addressType));
```

Now, go back and run **main.java** and at the bottom you'll see:

```
Address JSON:
{zip=20016, country=United States, street_1=345 Nonexistent
Avenue NW, city=Washington, street_2=null, state=DC,
type=Shipping}
```

Your Java code is now accessing the properties of your Kotlin code without getter or setter methods, and it's much cleaner to boot. Hooray!

Now it's time to see how you would set up things you'd normally use as `static` in Java.

"Static" values and functions from Kotlin

In Java, a `static` member in a class means that it can be accessed without an instance of the class. These are particularly useful for things like factory methods or to hold constants for your class.

Having used companion objects in Kotlin before, this probably sounds pretty familiar. But using companion objects from Java requires a little bit of help from the compiler.

In **Address.kt**, below the `JSONKeys` object but still within the `Address` class, add a new companion object with a single constant value:

```
companion object {
  val sampleFirstLine = "123 Fake Street"
}
```

In **main.kt**, add a line to print out this new value from the companion object:

```
println("Sample First Line: ${Address.sampleFirstLine}")
```

Run **main.kt**, and you'll see at the end of the console printout:

```
Sample First Line: 123 Fake Street
```

Nice — that was easy! Let's see how that works in Java. In **main.java**, add a similar line:

```
System.out.println("Sample first line of address: " +
Address.sampleFirstLine);
```

You'll immediately see an error, because nothing's told the compiler that Java should be able to see this particular value:

'sampleFirstLine' has private access in 'Address'

The error is slightly misleading - the property isn't `private`, but the compiler does need to know that it won't be changing in order to generate a static accessor.

Since `sampleFirstLine` is a simple `String` type, you can use the `const` keyword just as you did with the `JSONKeys` object in order to make it visible to Java.

Go to **Address.kt** and update the declaration to include `const`:

```
const val sampleFirstLine = "123 Fake Street"
```

Now, go back to **main.java**, and the error should have disappeared. Run **main.java** and you'll now see:

```
Sample first line of address: 123 Fake Street
```

Hooray! Now your Java code can access "static" variables on your Kotlin companion object. What about accessing a function which doesn't need an instance of the class?

Go back to **Address.kt**, and in the companion object, add a function that creates a sample Canadian address:

```
fun canadianSample(type: AddressType): Address {
  return Address(sampleFirstLine,
    "4th floor",
    "Vancouver",
    "BC",
    "A3G 4B2",
    type,
    "Canada")
}
```

Again, this is pretty straightforward to access in Kotlin. Go to **main.kt** and add the line:

```
println("Sample Canadian Address:\n$
{Address.canadianSample(AddressType.Billing)}")
```

Run **main.kt**, and you'll see printed at the end of the console:

```
Sample Canadian Address:
123 Fake Street
4th floor
Vancouver, BC A3G 4B2
CANADA
```

Now, go to **main.java**, and try to add something similar:

```
Address canadian = Address.canadianSample(AddressType.Shipping);
System.out.println(canadian);
```

Again, you'll see an error, although a slightly different one than you saw for the `val` in the companion object:

Cannot resolve method 'canadianSample'

Java can't see this method, but you can fix that using one of two approaches. You could update the call in Java to be:

```
Address canadian =
Address.Companion.canadianSample(AddressType.Shipping);
System.out.println(canadian);
```

Here, you're using the default `Companion` name of the Kotlin companion object to allow Java to access the method. Alternatively, you can fix the error with a simple annotation. Go back to **Address.kt**, and above the declaration of `canadianSample`, add a `@JvmStatic` annotation:

```
@JvmStatic
fun canadianSample(type: AddressType): Address { ... }
```

This tells the Kotlin compiler that, when generating Java code for the `Address` class, it should make `canadianSample()` a `static` method on the class for Java, and so you avoid needing to use the `Companion` name. Go back to **main.java**, and your error should now be cleared up. Run **main.java**, and at the bottom it'll print out:

```
123 Fake Street
4th floor
Vancouver, BC A3G 4B2
CANADA
```

Phew! You're now able to use both Kotlin code from Java and Java code from Kotlin. You can now go forth and conquer the Java Virtual Machine!

Challenge

For this chapter's challenge, you'll create an insecure way to store credit card information and access it from Java:

- Create a class in Kotlin for credit cards with properties for the card number, expiration month, expiration year and an optional security code (a.k.a., the CVV), which defaults to `null`. Make sure that you can use a constructor from Java, which takes advantage of the default value.
- Create a way to compare the current card to the passed-in card and determine if they are the same based on expiration year, expiration month and card number.
- Using one of the annotations you've already used in this chapter, suppress the creation of getter and setter methods for Java.
- Add a `List` of credit cards the user has stored to the `User` class.
- Add a function you can call statically from Java to validate whether or not an expiration date is valid (meaning, the expiration is in the future).
- Add an extension function on `User` to attempt to add a credit card to the user's list, but that rejects the card if it (a) is identical to another card or (b) has expired.
- Attempt to add these cards to the user's list of cards. How many cards does the user have when you're done?

> **Note**: In the real world, **do not** store credit card details like number/expiration/CVV, because that increases the probability of them getting stolen. Most payment processing companies have an API which will exchange this information for a token that represents the credit card. The token is very easy for the payment company to invalidate without having to cancel the underlying credit card should there be a security breach. The token is what you should generally store instead.

Key points

- Kotlin was designed from the beginning to be compatible with the JVM, and **Kotlin bytecode** can run anywhere that Java bytecode runs.
- You can intermix Kotlin and Java code within one project.
- It's possible to add Kotlin **extension functions** to classes written in Java, and also to call Kotlin **free functions** from Java code.
- Annotations like `@JvmOverloads` and `@JvmStatic` help you integrate your Java and Kotlin code.

Where to go from here?

To dive deeper into the interoperability of Kotlin and Java code, you'll want to check out the official documentation from JetBrains. If you're an Android developer, you'll also want to check out the interop guide created by Google:

- Android Kotlin Guide for writing cross-language code: https://android.github.io/kotlin-guides/interop.html

Like in life, no matter how careful you are as a developer, things will not always go as planned in your software. Next up, you'll see how you can handle unexpected conditions using Exceptions.